SAPIENZA
UNIVERSITÀ DI ROMA

ENGINEERING IN COMPUTER SCIENCE

Midnight Ghost Hunt

INTERACTIVE GRAPHICS REPORT

Professor:
Marco Schaerf

Student:
Niccolò Buonfiglio
1983405

Academic Year 2025/2026

Contents

1 Introduction

Midnight Ghost Hunt is a first-person shooter game set in a haunted graveyard. The player's objective is to eliminate all the ghosts in the map using a ray-gun. The game runs as a web application served on HTTP in the browser and is built as a set of ES modules.

2 Development Environment and Architecture

The project has been developed through the use of JavaScript modules for code organization. The development environment focuses around the use of Three.js, Ammo.js and Tween.js.

Three.js add-ons used:

Add-on	**Purpose**
`PointerLockControls`	First-person mouse-look and pointer lock API
`GLTFLoader`	Loading glTF/GLB 3D model files
`SkeletonUtils.clone`	Deep-cloning skeletal meshes for boss ghost instances
`EffectComposer`	Multi-pass post-processing pipeline
`RenderPass`	Base scene render pass
`UnrealBloomPass`	HDR bloom glow effect
`ShaderPass`	Custom vignette shader pass
`OutputPass`	Final tone-mapping/color-space output pass

Beyond the three external libraries, the code is split into modules, with main.js as the key focus:

Module	**Responsibility**
`main.js`	Booting the game up, updates to HUD and the rendering loop.
`World.js`	Procedural world creation and elements buildup.
`Player.js`	First-person controller, movement, and mechanics.
`Ghosts.js`	Ghost spawning, state machine, animations and visual on-screen effects.
`Physics.js`	Wrapper for Ammo.js.
`ProjectileManager.js`	Ray blasts trajectories and effects.
`Pickups.js`	Spawn and collection managing for power-up pickups.
`Effects.js`	Post-processing chain and particle system.
`AssetManager.js`	Asynchronous glTF/GLB loading and model normalization.
`Settings.js`	Graphics and difficulty presets.
`Sound.js`	Procedurally synthesized sound effects.
`Dev.js`	Developer overlay for debugging.

2.1 Imported 3D Models

External models are credited in `CREDITS.md`.

Asset	Purpose
Cage Lamp	Swinging cage subjected to physics
Tombstone Set	Set of different singular models for tombstones
Spooky Tree	Low polygon tree model
Human character	Player body
Wraith	Boss ghost 3D model
Ray Gun	First-person viewmodel
Fireball	Projectile

2.2 Textures

Texture	Use
Mud texture	Brownish color for the graveyard dirt
Tree texture	Bright color for the spooky spectral tree
Tombstone texture	Dark grey for the stone color

3 Gameplay logic

The game is structured in a simple way: the player spawns in the graveyard and has the objective to kill all ghosts in the map. The game either ends when all ghosts are eliminated or the player's health reaches zero. The player has a ray-gun, possesses a limited amount of ammo (which can be restored by touching pickups: either ray charges, blue bolt icon, or full recharges, yellow star icon), can shoot projectiles that have a maximum range (the game has an aiming assistance feature that glows green when aiming at a ghost within range, otherwise it's orange) and has visual indicators. There is a short cooldown between shots to prevent blast spamming by the player.

Game State Machine

`main.js` maintains a state variable with four values:

- menu: title screen visible
- playing: pointer locked and the full update loop is active
- paused: pointer is unlocked and the pause overlay shown. Update loop become idle
- over: game ended (either win or loss)

Transitions are driven by pointer-lock events, button clicks, and game conditions (all ghosts dead / player dead).

4 World Rendering

Rendering loop and configuration

The heart of the architecture is a single animation loop through `renderer.setAnimationLoop`. Each frame the elapsed time is measured with a Three.js clock and clamped to 50ms before being

used. The loop only runs the simulation while the state is playing and the pointer is locked. The THREE.WebGLRenderer is configured with antialiasing for smooth edges, switchable shadow quality, ACESFilmic tone mapping for cinematic lighting, sRGB color output, and quality-scaled pixel ratio.

4.1 Scene building

The world structure is characterized by its procedural generation and is responsible for the environment and its state.

The sky is generated procedurally at startup using a Canvas 2D dark blue gradient rendered into an equirectangular texture. The same texture is processed by `PMREMGenerator` to produce a pre-filtered environment map (`scene.environment`), which drives image-based lighting on reflective materials such as the fence and ray-gun.

The stars are 1500 particles distributed across the sky hemisphere using a spherical distribution. Each star is assigned an individual brightness and color temperature.

The moon is a sprite in the sky that emits a directional light that casts shadows across the entire graveyard and is the main focus for the shadow mapping. The directional light is positioned along the same direction vector as the sprite, and the shadow map covers a 25 unit frustum around the graveyard with configurable resolution and small bias parameters. The moon texture has a radial disk with an outer halo: the halo uses a lower opacity radial gradient giving the illusion of atmospheric scattering.

The playable area is a square centered on the origin, defined by a single exported constant. This constant is reused to keep the world dimensions consistent across modules.

Each tombstone variant extracted from the glTF is rendered as a `THREE.InstancedMesh`. All tombstones of the same variant share one draw call regardless of count, a performance optimisation that keeps draw calls low even with dozens of tombstones.

The fence system is built entirely from Three.js primitives and acts as a confine for the entire graveyard. The fence is constructed from distinct element types: vertical posts (cylinders with conical spikes), pickets (thin cylinders), and horizontal rails (thin boxes). Each side is tessellated with perSide=max(16,span/0.92) segments, where span=2×HALF, ensuring consistent spacing. All fence components except the rails use `InstancedMesh` to minimize draw calls. The material has `metalness: 0.95` and `roughness: 0.22`, making the fence pick up reflections from the environment map. There are physics colliders of box shape that prevent the player and ghosts from leaving the graveyard.

4.2 Smoke effect

Three layers of animated `THREE.Sprite` billboards simulate ground fog:

- Layer 0: Low, slow and faint sprites
- Layer 1: Mid-height with medium wind
- Layer 2: Higher and faster drift (lower number of sprites)

Each sprite drifts with a wind direction vector, oscillates vertically (undulation), sways left/right, breathes (scale pulses), and slowly rotates. A scattering approximation computes an alignment factor between each sprite's direction to camera and the moon direction, which means that sprites that lie between the camera and the moon glow with a scattered halo color, mimicking real moonlit fog. Sprites fade in based on distance from camera (it's invisible within 1.2 units) and wrap around the map boundary with modular arithmetic.

5 Physics

Physics is handled by **Ammo.js** and the `Physics` class (`js/Physics.js`) is able to wrap it into a minimal interface. World physics settings created in the Physics class include the Ammo.btDbvtBroadphase() and the Ammo.btSequentialImpulseConstraintSolver() and the gravity is managed by setGravity(new Ammo.btVector3(0, -9.82, 0)).

5.1 Rigid Bodies

Two types of bodies are used:

1. **Dynamic bodies (mass $>$ 0):** Used exclusively for the swinging cage lamp. It has a mass of 1.0 kg, linear damping 0.05, and angular damping 0.6. A `btPoint2PointConstraint` pins its top pivot to a fixed world point, making it act as a pendulum.
2. **Static bodies (mass = 0):** every other object.

5.2 Mesh Synchronisation and Step

The `linked` array in `Physics` stores $\{\text{mesh}, \text{body}\}$ pairs. Each `step()` call reads the rigid body's motion state and writes the position and quaternion back to the Three.js mesh. This keeps the visual representation exactly in sync with the physics simulation. `world.stepSimulation(dt, 4, 1/120)` is called every game frame.

6 Lightings and shadow mapping

6.1 Active lights

The scene uses five light sources, each with a distinct role:

Light	**Type**	**Role**
Moon	Directional Light	Main key light, casts shadows
Fill	Directional Light	Opposite side rim fill
Bounce	Directional Light	Ground bounce simulation
Hemisphere	Hemisphere Light	Ambient sky/ground gradient
Ambient	Ambient Light	Base lift to prevent pure black

6.2 Lightning

Every 30 seconds a lightning bolt strikes. It is added to the scene as a neon green `THREE.Line` with additive blending. A `DirectionalLight` fires a short intensity pulse sequence. Sound is triggered simultaneously.

7 Scene Construction

`World.js` builds the entire graveyard in its constructor and exposes an `update(dt, camPos)` method that the main loop calls every frame.

7.1 Torch Tree

The central feature of the map is a spooky dead tree with a hanging cage lamp. The tree is a glTF model scaled to 6.5 units high and placed at the map centre. The cage lamp is a separate glTF model, added as a child of a `THREE.Group` whose position is set to the branch tip pivot point $(-0.73, 3.2, -1.72)$. A physics rigid body is attached to the cage with a point-to-point constraint at its top making it a physically simulated pendulum.

When a ray-gun projectile hits the cage, `hitTorchCage(dir)` applies an impulse to the rigid body. `updateTorchCage()` enforces a maximum swing angle (1.2 rad) using velocity reflection and soft clamping, preventing the cage from flying over the top of the constraint.

7.2 Map Labyrinth

Tombstones are placed on a regular grid with random offsets and probability-based skipping. Placement logic avoids:

- The player spawn area (origin distance < 4)
- The map centre tree (centre distance < 3)
- Other tombstones that would overlap (circle-circle collision check with a 1.65 unit gap)

Each placed tombstone is assigned a random scale, yaw (full 360°), and occasional tilt (up to ± 0.14 rad). Tombstones also gain a proximity-based **tremor animation**: when the player approaches within 3.4 units, the tombstone begins to vibrate slightly. The intensity scales with proximity and uses high-frequency sin oscillation on the instance matrix that is updated via `setMatrixAt` every frame only for the animated instances.

7.3 Pickup System

Three pickup types float around the graveyard:

Type	Geometry	Color	Effect
Ray charge (`ray`)	Lightning bolt	Cyan	+1 ray charge
Full recharge (`full`)	Star	Gold	Fill ray charges to max
Soul fragment (`hp`)	Heart	Magenta	+1 life (capped at max)

Each pickup is a rotating emissive 3D icon with a matching point light, floating on a sine-wave bob animation. With the same logic as the tombstones spawning, pickups avoid the player spawn area, other pickups and other physical objects. Random polar coordinates are sampled up to 200 times; if no valid spot is found, a deterministic grid fallback is used.

When the player walks within 1.3 units of a pickup, it is collected: a scale-down tween plays, a particle puff fires, and the appropriate callback fires. After a randomized amount of time the pickup reappears at a new valid position with a scale-up tween (`Back.Out` easing for a bouncy pop-in).

8 Ghosts

`Ghosts.js` defines two entity classes: `Ghost` (individual ghost) and `GhostManager` (spawning, updates, damage).`ThreatAlarm` is a useful visual HUD element.

8.1 Normal Ghost build and trail

Each normal ghost is built from Three.js primitives:

- **Body:** A `LatheGeometry` profile rotated 360° to create a classic ghost silhouette
- **Core:** A smaller copy layered inside with additive blending
- **Aura:** A larger copy rendered from the back face for a halo
- **Eyes:** Two small `SphereGeometry` meshes placed on the upper front of the face

Materials use `THREE.AdditiveBlending` and a color that cycles between cyan and magenta, randomly assigned at spawn. A custom GLSL hook (`onBeforeCompile`) was written to fade the ghost's opacity toward its base, making it appear to trail off into transparency at the bottom.
Each ghost has a GPU point trail with 52 history positions stored in a `Float32Array`. Every frame the positions are shifted backwards one slot (like a ring buffer), and the newest position is inserted at index 0. Two custom per-vertex attributes (`alpha`, `pSize`) control each point's transparency and size: both also use `onBeforeCompile` to inject the custom GLSL. The result is a smooth blended tail that fades and narrows toward the end.

8.2 Blob Shadow

Each ghost casts a blob shadow: a flat `PlaneGeometry` placed just above the ground, textured with a radial gradient. Its position is offset in the direction of the moon light to simulate a cast shadow. Normal ghosts also have two smaller trail blobs spaced along their recent history path.

8.3 Wraith Build

Bosses use the imported `wraith.gltf` skeletal model, cloned with `SkeletonUtils.clone` to preserve the skeleton. Their material is modified to add emissive glow. An arm-bone animation system reads the upper-arm, lower-arm, and hand bones by name and interpolates them from rest to a "reaching" pose using slerp. The wraith stretches its arms forward when chasing the player, then relaxes them when patrolling.

When a wraith is hit, it enters an `enrageHit` state: its scale grows (tween with `Back.Out` easing), its emissive intensity increases, its point light radius expands, and a pain-flash animation plays. Each hit makes the boss slightly larger and more intense.
When not chasing, the wraith occasionally performs a full 360° rotation animation (spin cycle every 3.5+ seconds).

8.4 Ghost AI

Each ghost runs a simple state machine per frame:

1. **Recoil:** Has just been hit -> zero speed, brief stagger
2. **Pacific mode:** Wander to random targets (ghost cannot harm the player)
3. **Player invincible:** Retreat away from player position
4. **Chasing:** Player within detection radius and not invincible -> charge at full chase speed
5. **Patrolling:** Outside detection range -> move toward a random zone target

Detection radius varies by difficulty (8–10 units). Boss ghosts detect 2 units farther and move 15% faster than normal ghosts. Ghost-to-ghost separation is resolved every frame with a simple pairwise circle-circle push-apart loop, preventing ghosts from stacking on top of each other.
Obstacle avoidance uses the same circular collider list as the player: tombstones and the tree are circle obstacles that ghosts are pushed away from.

8.5 Threat Alarm

When an off-screen ghost is chasing the player and within 15 units, `ThreatAlarm` draws pulsing SVG arcs directly on the DOM (overlaid on the game canvas). The arc angle is computed from the ghost's direction relative to the camera's forward vector. Three concentric arcs with decreasing stroke width and opacity create a depth effect. Color intensity increases as the ghost gets closer.

9 Player logic

`Player.js` encapsulates the first-person controller.

9.1 Movement

`PointerLockControls` handles mouse-look. The player holds position at a fixed eye height of 1.7 units (`EYE = 1.7`). Movement is computed from keyboard input (WASD + arrow keys) by projecting the camera's forward and right vectors onto the XZ plane and summing them. Speed is constant at 2.9 units/s (walking).
Collision resolution is a simple iterative circle push: for each circular collider (tombstones, tree trunk), if the player centre is closer than collider.radius + PLAYER_R (0.45), push the player outward along the overlap axis.
The player is also clamped to the map boundary $[-\text{HALF} + 0.6, \text{HALF} - 0.6]$ on both axes.

9.2 Shooting Mechanics and Aim Indicator

When the player left-clicks:

1. Cooldown (0.5 s) is checked
2. Invincibility period suppresses firing
3. Ray charge is decremented
4. A recoil tween pushes the gun model 0.12 units back in Z, then smoothly returns it
5. A `THREE.Raycaster` is cast from the screen centre
6. The raycaster tests ghost hit meshes and solid world meshes in order
7. If a ghost is hit closer than the nearest solid, a projectile is spawned homing on that ghost; otherwise it flies toward the hit point or max range

Every frame, `aimGhost()` casts a raycaster silently. If a ghost is in the crosshair and not blocked by geometry, the crosshair CSS class changes to `aim` (a subtle color shift indicates a valid target).

9.3 Invincibility and Knockback

After taking a hit the player gains 3 seconds of invincibility (`INVINCIBLE = 3.0`). During this window ghosts cannot deal damage and shooting is suppressed. A UI bar drains over the 3 seconds to indicate how long protection lasts.
On hit, the player also receives a knockback impulse away from the ghost: a velocity is applied over 0.34 seconds, pushing the player in the opposite direction of the ghost's position.

9.4 Player Body

A human skeletal model is loaded and placed behind the camera (offset 0.18 units back). It is rendered with a fully transparent material so it acts as a shadow-casting silhouette only. The player can see their own shadow on the ground without a visible body occluding the view. Arm bones are partially animated: the right arm follows the camera orientation so the gun hand tracks mouse movement.

9.5 Ray-Gun Viewmodel

The ray-gun glTF model is attached directly to the camera as a child node, positioned at $(0.32, -0.30, -0.55)$ in camera space (lower right). Its material is made slightly reflective (`metalness: 0.5`, `roughness: 0.45`) to pick up the environment map. The gun is hidden in menu mode and shown when gameplay starts.

10 Projectile System

`ProjectileManager.js` manages the ray-gun projectile.
The fireball glTF mesh is cloned for each shot. Its material is replaced with a bright additive-blended green. A `PointLight` is attached to the projectile so it lights up nearby geometry as it flies.

10.1 Flight

Projectiles travel at 40 units/second. If the shot was aimed at a ghost (`homingTarget`), the projectile continuously steers toward the ghost's current position, even if the ghost moves after the shot. This gives the ray a mild homing characteristic. If the ghost dies before impact, the projectile continues to the last recorded position.

10.2 Hit Resolution

When the projectile reaches its destination (within step + 0.35 units):

- **Ghost hit:** `ghostManager.damage()` is called. If the ghost dies, the player refunds 1 ray charge (3 for a boss kill). An explosion effect plays.
- **Miss:** A fizzle effect plays, the projectile turns red and shrinks with a brief red flash.

10.3 Explosion Effect

On a ghost kill, three overlapping particle puffs of different green shades and spreads fire simultaneously. Two expanding translucent spheres grow outward using TWEEN then dispose. A `PointLight` flashes at the impact point and fades over 300 to 460 ms. Boss kills produce a proportionally larger explosion.

11 Post-Processing Effects

The scene is rendered through an `EffectComposer` pipeline in `Effects.js`:

```
RenderPass -> UnrealBloomPass -> ShaderPass (vignette) -> OutputPass
```

11.1 Bloom

`UnrealBloomPass` applies HDR bloom: bright pixels (above a configurable threshold) are blurred and additively composited back. This causes the ghosts, projectiles, pickups, and lightning to emit a soft glow halo. Bloom strength, radius, and threshold are configurable via the Graphics settings panel (bloom can be disabled entirely on Low quality).

11.2 Vignette

A custom GLSL `ShaderPass` darkens the screen corners:

```
vec2 uv = (vUv - 0.5) * offset;
float v = clamp(1.0 - dot(uv,uv) * darkness, 0.0, 1.0);
gl_FragColor = vec4(tex.rgb * v, tex.a);
```

The darkening is radially symmetric, drawing the eye to the screen centre. The default values (`offset: 1.15`, `darkness: 1.25`) produce a subtle but atmospheric result.

11.3 Particle System

A shared GPU particle pool of up to 900 particles is managed in `Effects`. All particles share a single `THREE.Points` object, avoiding per-particle draw calls. Custom per-vertex `alpha` and free slots are tracked in `Float32Arrays`. When a ghost is killed or a pickup collected, `puff()` takes slots from the free list, sets their position, velocity, and color, then returns them to the free list as they expire. Particles rise gently (constant +0.4 m/s upward drift) and decelerate.

11.4 Hurt Flash

When the player gets damaged, there is the trigger of a brief red overlay. The class is removed and re-added in the same frame using `void offsetWidth` to force a browser style recalculation, ensuring the animation restarts even on rapid successive hits.

12 Settings and Difficulty

12.1 Graphics Quality Presets

Setting	**Low**	**High (default)**
Render scale	0.70×	1.00×
Shadow map	512 px	2048 px
Bloom	Off	On
Particles	50%	100%
Shadow type	Basic	PCFSoft

Switching preset rebuilds the renderer pixel ratio, shadow map, and bloom configuration at runtime.

12.2 Difficulty Levels

Parameter	**Easy**	**Medium**	**Hard**
Starting lives	4	3	2
Starting ray charges	6	5	4
Max ray charges	10	8	7
Normal ghosts	5	8	11
Boss ghosts (Wraiths)	1	1	2
Ghost speed multiplier	×0.85	×1.00	×1.20
Detection range	8 units	9 units	10 units
Ray charge pickups	6	5	4

12.3 Pacific Mode

A special mode toggled on the main menu. In Pacific mode ghosts wander peacefully and never attack. The player receives extra ray charge and lives. Intended for exploring the graveyard or demonstrating the visual effects.

13 User Manual

13.1 Main Menu Options

- **ENTER THE GROUNDS:** Start hunting immediately with current settings.
- **RULES:** Read the objectives and controls summary.
- **GAMEPLAY:** Choose difficulty (Easy / Medium / Hard). The description beneath each button shows the number of spectres, wraiths, and soul fragments.
- **GRAPHICS:** Choose Low or High graphics quality. Low reduces shadow resolution and disables bloom for better performance on integrated graphics.
- **Pacific mode:** Toggle this on to explore the graveyard without being attacked.

13.2 Controls

Input	Action
W / Up Arrow	Move forward
S / Down Arrow	Move backward
A / Left Arrow	Strafe left
D / Right Arrow	Strafe right
Mouse	Look around
Left Click	Fire the ray-gun
Esc	Pause / release pointer

Clicking on the canvas requests pointer lock. The mouse cursor disappears and is captured for first-person look. Press **Esc** to release.

13.3 HUD Guide

- **SOUL hearts:** Each filled heart is one life. Emptied hearts are lost. When all are lost the game ends.
- **RAY CHARGE pips:** Each filled pip is one ray shot available. When all are spent, find a pickup.
- **SPECTRES counter:** The number of spectres remaining. Reduce to 0 to win.
- **Crosshair:** Turns red when out of ammo. Shows a cooldown arc after each shot. Glows when aimed at a valid ghost.

13.4 Win and Loss Conditions

- **CLEANSED (Win):** All spectres are dissolved. A victory screen appears.
- **CONSUMED (Loss):** All soul fragments are lost. A game-over screen appears.

From either screen, **HUNT AGAIN** restarts with the same settings; **MAIN MENU** returns to the title screen.